AC4A: Access Control for Agents

RESHABH K SHARMA, University of Washington, USA
DAN GROSSMAN, University of Washington, USA

Large Language Model (LLM) agents combine the chat interaction capabilities of LLMs with the power to interact with external tools and APIs. This enables them to perform complex tasks and act autonomously to achieve user goals. However, current agent systems operate on an all-or-nothing basis: either an agent has full access to an API's capabilities or a webpage's content, or it has no access at all. This coarse-grained approach forces users to trust agents with more capabilities than they actually need for a given task. For instance, when asking an agent to book a flight to a specific destination, the user must grant it access to the entire flight booking API, which could allow the agent to book flights to any destination, modify existing bookings, or access personal payment information.

In this paper, we introduce *AC4A*: an access control framework for LLM agents. As agents become more capable and autonomous, users need a way to limit what APIs or portions of web pages these agents can access eliminating the need to trust them with everything an API or web page allows. Our goal with *AC4A* is simple: to provide a framework for defining permissions that lets agents access only the resources they need. *AC4A* works across both API-based and browser-based agents. It does not prescribe what permissions should be, but offers a flexible way to define and enforce them, making it practical for real-world systems.

AC4A works by creating read or write access permissions over resources, drawing inspiration from established access control frameworks like the one for the Unix file system. We represent resources as hierarchies and use them to create permissions that are easier to understand and generate. The framework comes with a permission management dashboard that can be used to create, edit, and monitor permissions. These permissions can either be created manually or using an LLM. During runtime, the agent's execution is intercepted and access to a resource is only allowed if there is a permission allowing it. We evaluate the effectiveness of *AC4A* through case studies demonstrating the use of *AC4A* to enforce permissions over real-world websites and APIs accessed by agents. The source code of *AC4A* is available at https://github.com/reSHARMA/AC4A.

1 INTRODUCTION

Large Language Models (LLMs) have evolved from passive question-answering systems to active agents that can interact with the external world through APIs and web browsers [32]. These agents can now perform complex tasks such as booking travel, managing calendars, or writing code by calling external services and navigating web interfaces [1, 3]. While this autonomy enables powerful applications, it also creates new security challenges that existing systems are ill-equipped to handle.

Current LLM agents lack an access control framework unlike traditional software systems. While access control systems themselves cannot protect against all security threats, they serve as essential components that enable other security mechanisms. Current agentic systems operate on an all-or-nothing basis: either an agent has full access to an API's capabilities or a webpage's content, or it has no access at all. This coarse-grained approach forces users to trust agents with more capabilities than they actually need for a given task. For instance, when asking an agent to book a flight to a specific destination, the user must grant it access to the entire flight booking API, which could allow the agent to book flights to any destination or modify existing bookings. Similarly, when using a browser-based agent to check a calendar, the agent gains access to the entire webpage, including sensitive information such as other calendar entries and personal details visible to the agent.

The lack of access control is distinct from other security challenges for agents. There are many important security concerns related to agents and the LLMs used underneath like hallucination [14],

Authors' addresses: Reshabh K Sharma, University of Washington, Seattle, Washington, USA, reshabh@cs.washington.edu; Dan Grossman, University of Washington, Seattle, Washington, USA, djg@cs.washington.edu.

prompt injection attacks [15, 27], and malicious instruction hiding [33, 34]. Unlike defenses to these, an access control framework cannot mitigate them directly similar to how vulnerabilities in traditional systems such as buffer overflows, side-channel attacks, and phishing cannot be addressed by access control systems alone. However, it is unimaginable to think about a secure system without an access control system. We focus on controlling what resources the agent can access in the first place, which we believe is a prerequisite for addressing other security concerns.

We introduce *AC4A*, an access control framework for LLM agents. *AC4A* provides fine-grained permissions that allow agents to access only the specific resources they need for a given task, rather than requiring users to trust them with broad access to entire APIs or web pages. Our framework works across both API-based and browser-based agents, using a unified permission infrastructure that can be extended to support new types of agent implementations in the future.

AC4A operates by creating read or write access permissions over resources in a hierarchical structure, drawing inspiration from established access control systems such as Unix [21, 23, 25]. We represent resources hierarchically and use this structure to create permissions that are both intuitive and enforceable. The framework includes a permission management dashboard (Figure 11) for creating, editing, and monitoring permissions, which can be generated either manually or using an LLM. During runtime, agent execution is intercepted and access to resources is granted only when there is an explicit permission.

In Section 3, we discuss the design of *AC4A* in detail. We demonstrate the working of *AC4A* by creating permissions to protect game data from an agent designed to interact with a tic-tac-toe application using its APIs and webpage.

Overall, we make the following contributions:

- An access control framework for different agent implementations: *AC4A* is the first framework of its kind to perform access control over browser-based agents using the same permission infrastructure as API-based agents. It provides a consistent way to define permissions across different resource types by representing them in a hierarchical structure, making permissions portable and easier to understand.
- Comprehensive case studies demonstrating practical deployment: We evaluate *AC4A* on real-world scenarios including flight booking APIs and web pages such as Outlook Calendar and Expedia, showing that agents can successfully work within restricted permissions, request additional permissions when needed, and complete tasks while respecting access controls.
- A permission management system with LLM-assisted generation: *AC4A* includes a dashboard for creating and managing permissions, with the ability to generate permissions using LLMs, making the framework practical for real-world deployment.

2 THREAT MODEL

The primary goal of our access control framework is to enforce a given set of permissions for agents. Our threat model is intentionally scoped to the access control layer: we assume that the agent is untrusted, while the access control framework itself and the permissions it receives are trusted. The framework's guarantees are strictly limited to the correct enforcement of the permissions it is given.

2.1 Untrusted Principals

2.1.1 Agents and Underlying LLMs. The agent, which may be authored by third parties, is the central untrusted entity. Regardless of its origin or intent, an agent may be compromised or designed

with malicious intent. Similarly, the underlying LLMs are not trusted; they may contain biases, implementation errors, or backdoors.

2.1.2 APIs and Application Providers. APIs exposed by various applications are essential for agent operation. We trust these applications to provide accurate permission signatures that correctly map their API endpoints to the data they expose. For example, we trust that an application correctly identifies a write API endpoint as requiring write permissions. What these applications do with the data once it reaches them is beyond our control and outside the scope of our access control framework, just as the Unix file system implementation is trusted to not leak or corrupt data.

2.1.3 Web Content. The content of web pages is only partially trusted within our model [19, 28]. We do not trust the content on the web pages. Web content can manipulate agents into accessing unintended data. Permission configurations derived directly from web content without rigorous validation are considered inherently compromised. Like APIs, our framework requires a mapping of webpage elements to the data they expose. For this, we must trust the annotations provided by the web page (if any) or rely on external mappings, both of which may be imperfect.

2.1.4 End Users. End user input is only trusted when it is involved in the creation or modification of permissions.

2.2 Trusted Components

The access control framework and the monitoring system are trusted. These components are responsible for expressing, enforcing, and auditing permissions, as well as for extracting permission signatures from API calls and web elements. Permission-granters, defined as individuals with the authority to create or modify permissions, are fully trusted within our model. In practice, this role may be filled by end users themselves, supervisors in corporate settings, or system administrators. A compromised permission-granter represents an inherent limitation that is outside the scope of our threat model. Finally, the permissions provided to the framework are trusted and considered the source of truth. If the permissions are flawed, this is outside the scope of our enforcement model.

2.3 Scope and Limitations of Access Control

It is critical to understand the scope and limitations of our approach. Access control is a **necessary but insufficient** tool for achieving higher-level security properties. Our framework's role is not to guarantee that all unintended access to resources is prevented, but rather to rigorously enforce the restrictions specified in the provided permission set. If a necessary restriction is omitted or misconfigured by a trusted author, the framework cannot prevent resource misuse.

3 DESIGN

AC4A is an access control framework designed to enforce fine-grained permissions over resources accessed by autonomous agents. As agents operate autonomously and can utilize various tools to both retrieve and modify resources through multiple channels (APIs, web browsers, etc.), implementing a unified access control mechanism becomes critical to govern resource availability regardless of the access method. Permissions are the core of *AC4A*. Well-defined permissions are crucial for preventing unintended resource access and ensuring that agents operate within their authorized boundaries. A permission in *AC4A* is made up of a resource and an action. A resource is defined by a resource type and a resource value and an action is defined by one of the action types. *AC4A* does not have any knowledge of the resources and their action types. The applications are responsible for defining their own resource types and action types. *AC4A* provides infrastructure to

define these resources and actions, create permissions for them and also enforce these permissions at runtime.

3.1 Resource Modeling

AC4A treats resources as completely opaque entities without any knowledge of their possible values or how they are modeled. Consequently, the system imposes no constraints on the definitions of resource types or their associated values. Instead, it requires applications to model their own resource type nodes using the interfaces provided by *AC4A*.

In *AC4A*, every resource value must be associated with a specific resource type. A valid resource type is defined as a directed path within the tree of resource type nodes, always originating from the root. Therefore, a resource type tree containing only a single resource type node represents a singular, flat resource type.

A resource type node can be an enum representing non-overlapping resource values. For example, an enum-like resource type node *Day* defined by a calendar application represents resource values as calendar events for a specific day (e.g., Monday, Tuesday). Resource type node can also represent overlapping resource values. For instance, a calendar application can define a resource type node as UNIX timestamp intervals such that the resource values are the calendar events within the intervals, which can be overlapping.

In many cases, the resource type nodes themselves overlap, such that multiple resource type nodes can represent the same resource value. For instance, a calendar event for *October 2025* exists within the resource value space of a *Month* resource type node with the value October and also in a *Year* resource type node with the value 2025. These natural structured relationships between resource type nodes are allowed by *AC4A*, forming a parent-child relationship even if there is only a partial overlap in the resource value space. Such partial overlaps can be used to model resources with higher granularity. For example, a resource type node representing credit card data and another node representing the card number can have a parent-child relationship because they have a partially overlapping credit card number in the resource values, even when they do not represent the exact same resource value space.

The parent-child relationship between the resource type nodes results in a resource type tree, and any directed path from the root constitutes a valid resource type exposed to the framework. A calendar application, for instance, might define resource type nodes as *Year*, *Month*, and *Day*, and establish relationships between them such that *Year* is a parent of *Month* and *Month* is a parent of *Day*, resulting in a resource type tree. Any directed path starting from the root is a valid resource type that the application can choose to expose. In this case, valid types could be *Year, Month, Day*, *Year, Month*, and *Year*.

AC4A also supports recursive relation between resource type nodes resulting in an infinite possible paths from the root that represent a valid resource type. Such recursive modeling is used in cases like a UNIX file system, which can be considered to be made up of only two resource type nodes, directory and file, such that both directory and file are children of the directory node. Although there are infinite possible resource types for this modeling, the application must list the resource types it supports, which are always finite.

Resource values are always explicitly written with their resource types using the format `Type(Val)`. If a resource type is composed of multiple resource type nodes using the parent-child relation, the values are chained using the '::' operator, such as `Type1(Val1)::Type2(Val2)`.

The applications can also provide resource type descriptions, semantic meaning in natural language, and representative examples of resource values. These additional descriptions are optional and primarily used to improve LLM-based permission generation accuracy.

3.2 Access Control Model

AC4A uses a simple access control model where each application defines its own set of actions as an enum unlike resources they do not have any structure and are assumed to be non-overlapping. Most applications will likely have **read** and **write** actions, but applications are free to define additional actions as needed. For example, a calendar application might define a **create** action that is less permissive than write access as it allows the creation of new events without being able to modify or delete existing ones.

The applications could simply use `{read, write}` as the action enum. Even though this classification is admittedly simple and not particularly precise, it works adequately for most applications. `Read` covers any operation that makes resources visible to an agent without modifying the underlying resource store. This includes resource retrieval and querying operations, search functionality, resource display and presentation, and read-only API calls. **Write** access covers operations that modify the internal state of resource systems, including resource modification and deletion. The framework only requires that when checking permissions, the action types match exactly between what the agent has permission for and what the agent needs. If an agent has `read` permission but needs `write` permission, the access is denied regardless of the specific resources involved.

This approach gives applications complete control over their access control semantics while providing a simple, unified interface for permission checking across different applications.

3.3 Permission System

Permissions in *AC4A* are pair of a resource value and an action. Each permission grants the specified action on the specified resource value.

3.3.1 Permission Format. A permission is a pair of two components:

- **Resource Value**: Resource value as defined by the application defined resource type.
- **Action**: An action from the application's action enum

Definition 3.1 (Permission Resource Syntax). The abstract syntax of the *Resource* structure is defined as follows:

$$\text{Resource} ::= \text{name} \ \texttt{(} \ \text{value} \ \texttt{)} \ \{\texttt{::} \ \text{Resource}\} \tag{1}$$

$$\text{value} ::= \texttt{?} \mid \text{string} \tag{2}$$

Where name represents the resource type node and string represents the value of the resource. The resource value can also be a wildcard ? that matches any value of the resource type node or it can be a specific string value. If a permission has a resource value that has wildcard ? at any position then it is considered to be a set of pairs of all the resource values and action that matches that regex. The `::` operator is used to define the parent-child relationships of the resource types as defined by the application.

Table 1 presents examples of permissions made from resource values and action pairs for different applications.

3.4 Permission Checking

AC4A enforces permissions by checking whether the resources accessed by the agent are allowed based on the active permissions. Given a set of resource and action pairs that are needed by the agent for an access, *AC4A* performs permission checking for each needed action type. *AC4A* checks if the resources granted from the active permissions covers the resources needed by the agent. The access is only allowed when need becomes empty at the end for each action type in the need set. The resource are checked separately for each action type ensuring that permissions of one

Resource Type and Value	Access	Description
`Calendar:Year(2025)`	`write`	Grants access for writing to resource of type Calendar:Year resource and value 2025.
`Wallet:CreditCard(Gas) ::Wallet:CreditCardNumber(?)`	`read`	Grants access for reading all the resources of type Wallet:CreditCardNumber that are in the resource of type Wallet:CreditCard and value *Gas*.
`Cookie:Essential(?)`	`create`	Grants access for creating any resource of type Cookie:Essential.

Table 1. Examples of permissions made up of resource type and values from the resource modeling along with access mode.

action type cannot satisfy the needs of another. The difference between the resources granted and the resources needed by the agent is provided by a resource difference function defined by each application.

3.4.1 Resource Difference Function. *AC4A* unifies permission reasoning around a single primitive supplied by each application: a *resource_difference* function. When an agent requests access to some resource, we need to determine whether the agent's currently granted resources are sufficient. Since an agent may have access to multiple resources and each resource may cover different parts of the needed resource space, we iteratively check what remains unmet after accounting for each granted resource.

Each application implements a `resource_difference` function with the following signature:

$$\text{resource_difference}(\mathit{Need}, \mathit{Have}) \Rightarrow \mathit{Remaining}$$

where *Need* and *Have* are resources, and *Remaining* is a finite set of resources whose union denotes what part of *Need* is still unmet after accounting for *Have*.

Given a set of resource and action pairs that are needed by the agent. *AC4A* performs permission checking for each action type from the need set. Let $Need$ be the required resources for the action type $Action$ from the set of needed actions, $NeedActions$ and let $HaveResources$ be the set of granted resources from the active permissions with action type $Action$. *AC4A* then uses resource difference function to check if the resources granted from the active permissions covers the resources needed by the agent. *AC4A* maintains a working set of unmet resource requirements. Starting with the needed resource, we iteratively subtract what each granted resource covers until either nothing remains (need met) or we exhaust all granted resources (need unmet). The access is only allowed when need becomes empty at the end for each action type in $NeedActions$. Algorithm 1 formalizes this approach where *getResources* returns a list of resources associated with a specific action.

Algorithm 1 assumes that the `resource_difference` function provided by the applications is sound. This means it does not allow access to unauthorized resources. To achieve this, the function need not precisely return the exact remaining resources after accounting for granted resources. As long as it never returns resources outside of what was originally needed, the algorithm remains sound. For example, if an agent needs access to `{tag1, tag2, tag3}` and have resource `{tag1}`, the `resource_difference` function could return either `{tag2, tag3}` (precise) or `{tag1, tag2, tag3}` (conservative). Both implementations are sound since they do not grant access to anything beyond what was needed.

Algorithm 1 *AC4A*'s Permission Checking Algorithm

```
for Action ∈ NeedActions do
    HaveResources ← getResources(ActivePermissions, Action)
    Remaining ← {Need}
    for each have in HaveResources do
        Next ← ∅
        for each n in Remaining do
            Next ← Next ∪ resource_difference(n, have)
        end for
        Remaining ← Next
        if Remaining = ∅ then
            break
        end if
    end for
    if Remaining ≠ ∅ then
        return false
    end if
end for
return true
```

Order independence means that the granted resources can be processed by *AC4A* in any order without affecting the final decision. *AC4A* does not guarantee order independence of the `resource_difference` function, but it is the application's responsibility to ensure that the function is implemented in a way that maintains this property.

Consider an application that models resources as time intervals. If the `resource_difference` function is implemented to precisely compute interval subtraction, it will be order independent. However, if the function sometimes over-approximates coverage (e.g., rounding up intervals), different orderings of granted resources could produce different final Remaining sets, breaking order independence. For instance, if `resource_difference([9:00-17:00], [9:00-12:30])` incorrectly returns `[13:00-17:00]` (rounding up) but processes other intervals precisely, different orderings of granted resources could produce different final Remaining sets. This inconsistent precision breaks order independence, though the algorithm remains sound.

Applications should think carefully about whether their `resource_difference` implementation behaves consistently regardless of the order in which granted resources are processed in Algorithm 1. Set-based resources naturally achieve this property. Hierarchical and interval-based resources require consistent refinement or conservative handling. Applications can choose their precision-performance trade-off but should ensure their choice does not introduce order dependence and remains sound. *AC4A* provides support for common patterns like conservatively finding the remaining resources for enums, intervals and trees making the implementation of the resource_difference function easier. These helper functions are not aware about the actual data represented by the resource types and only handle simple cases like when the need is a subtree in have, enum value of need and have are the same. Applications are free to ignore them and implement their own resource_difference functions for more complex scenarios like overlapping resource types or non-hierarchical relationships. For example, a calendar application might want to model both temporal resources (dates, times) and functional resources (meetings, reminders, all-day events) that overlap in complex ways.

In such cases, the resource_difference function can handle these relationships in different ways:

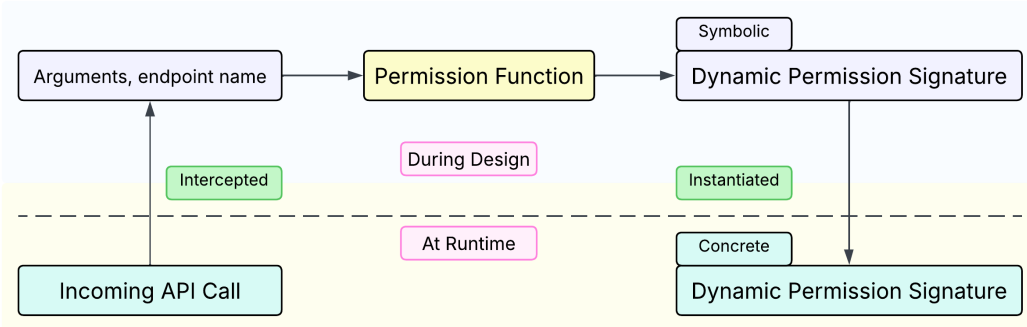

Fig. 1. Dynamic permission signature life cycle: API developers write permission functions using symbolic argument values, then at runtime, API calls are intercepted and the permission function instantiates dynamic permission signatures with concrete argument values to determine required permissions.

- **Conservative approach**: If one have access to resource `Calendar:Reminders(*)` and needs resource `Calendar:Year(2025)::Calendar:Month(2)`, the function might return need for resource `Calendar:Year(2025)::Calendar:Month(2)` treating the dimensions as incomparable.
- **Precise approach**: The function might compute that you need all types of resources for February 2025 except reminders, providing more granular access control.

Both approaches are valid and the choice is entirely up to the application. The framework only requires that the resource_difference function is sound and does not violate order independence.

3.5 Dynamic Permission Signature

In *AC4A*, permissions are simple (resource value, action) pairs, but to enforce these permissions, we need to determine what resource and action are required for each access. Similar to how a function signature describes how a function must be called, including arguments and their types, a dynamic permission signature represents the minimum permissions required for a given access to proceed. A dynamic permission signature is simply a conjunction of (resource value, action) pairs that represents what the agent needs permission for. For example, a request to get calendar events for January 2025 might require the permission `(Calendar:Year(2025)::Calendar:Month(1), read)`, while creating a new event might require `(Calendar:Year(2025)::Calendar:Month(1), create)`.

The system authorizes an access if and only if the agent has a permission that covers the required resource and action. This is determined using the application's `resource_difference` function if it returns an empty set and the action types match, then the access is authorized.

3.5.1 Permission Function. Dynamic permission signature extraction for APIs and web pages requires a permission function to translate API endpoints along with their arguments and web page elements to resource types and values along with access types during runtime. Since dynamic permission signatures are actual minimum permissions with concrete values, the permission function generates these signatures with symbolic values for the arguments and web elements, which are then instantiated at runtime with actual values to produce the final dynamic permission signature.

API Endpoint Mapping. Each application can define a permission function to map API endpoints and their arguments to resource types and values along with access types during runtime. This

API Endpoint / Web Element	Dynamic Permission Signature
get_calendar_events(start_date: 2024-01-01, end_date: 2024-01-31)	Calendar:Year(2024) ::Calendar:Month(1)::read
create_workspace_button	Trello:Workspace(?)::create

Fig. 2. Examples of dynamic permission signatures showing the minimum permission required for a given access to proceed. These signatures are generated by the permission function and are instantiated at runtime with the actual values of the arguments and the web element.

function is responsible for determining what resources and actions are needed for a given request represented as a dynamic permission signature. This is used by the `resource_difference` function to determine if the agent has the required permissions to access the desired resources. Figure 3 shows an example of API mapping for Calendar, demonstrating how different endpoints are mapped to resources and actions based on their arguments.

```
def permission_function(kwargs, endpoint_name):
    resources = []
    start_time = kwargs['start_time']
    duration = kwargs['duration']
    end_time = start_time + duration
    month_names = ['January', 'February', 'March', 'April', 'May', 'June',
                   'July', 'August', 'September', 'October', 'November', 'December']
    def get_hierarchy(time_value):
        return [
            (365, 'Year', time_value.year),
            (30, 'Month', month_names[time_value.month - 1]),
            (1, 'Day', time_value.day),
            (0, 'Hour', time_value.hour)
        ]
    for (_, label, start_value), (_, _, end_value) in zip(
        get_hierarchy(start_time), get_hierarchy(end_time)):
        resources.append(f'Calendar:{label}({start_value})')
        if start_value != end_value:
            break
    access_type = (
        "create"
        if any(k in endpoint_name for k in ["reserve", "create", "add"])
        else "write"
        if any(k in endpoint_name for k in ["update", "edit", "modify"])
        else "read"
    )
    return '::'.join(resources), access_type
```

Fig. 3. Example API mapping for calendar showing how different endpoints are mapped to resources based on their arguments and access types. The function generates dynamic permission signatures by mapping endpoint names and arguments to specific data types in the hierarchy. The function takes the endpoint name and argument values as input and returns the resource and access type computed based on the arguments and the endpoint name.

Web Page Element Mapping. We introduce a method to map web page elements to resources and actions. We create configuration files for each web page that map CSS selectors to resources and actions. This mapping is not straightforward due to the following challenges:

(1) HTML can be dynamically generated using JavaScript.
(2) CSS selectors using IDs or class names are unreliable because web pages randomize them to protect against data scraping.

The configuration file for each web page is a JSON file with keys representing resources and values being lists of HTML page elements as CSS selectors. These are extended CSS selectors that support complex queries based on data, such as `button[aria-label*='select to change the month']` or `span:contains('New event')`. We need this extension because IDs and class names are often dynamically generated by backends and cannot be reliably used for consistent mapping.

There are cases where a unique HTML element represents multiple values of the same resource type based on the context. For example, a calendar might show multiple years, or a flight might show multiple destinations. To handle this, we allow defining data values dynamically, making the configuration key dynamic.

Figure 4 shows a more comprehensive example of web mapping for Outlook calendar, demonstrating how different web page elements are mapped to the resource hierarchy, including both static and dynamic data value mappings.

Dynamic Resource Value Definition. To handle cases where HTML elements represent dynamic resources, we introduce a dynamic resource value definition syntax. The syntax is defined as:

$$\$data\{\text{css selector}\}\{\text{list transformation}\}[\text{Index}]\{\text{value transformation}\}@\text{attr}$$

This extracts data for a provided CSS selector and its attribute, which must be a valid HTML attribute: `value`, `href`, `src`, `alt`, `title`, or `text` (the text inside the element referenced by the CSS selector). The data is first transformed by the list transformation, which takes the data extracted by the CSS selector as input and converts it to a list of strings. The index selects a string from that list, which can be further transformed by the value transformer. Multiple transformations can be chained together.

$$\text{Calendar:Year}(\$data\{\text{input[aria-label='Start date']}\}\{\texttt{split_slash}\}[2]@\text{value})$$

This CSS selector points to an input element with a value like MM/DD/YYYY, which is first converted to a list by the `split_slash` transformation (splitting on the '/' character), then the value at the second index (the year) is returned and used as the value for type `Calendar:Year`.

Mapping configurations can be developed publicly, maintained, and audited by the community. These configuration files enable *AC4A* to function without depending on website owners for custom annotations, simplifying dynamic permission signature extraction. We also propose an annotation schema that web developers can use to make *AC4A* work for their web pages without configuration files, allowing dynamic permission signatures to be extracted directly from HTML.

The annotation schema uses the following HTML attributes:

(1) `data-ac4a-resource` (resource type and value)
(2) `data-ac4a-action` (action type)
(3) `data-ac4a-static` (true/false)

The `data-ac4a-resource` attribute represents the resource made up of a resource type and a value, and the `data-ac4a-action` attribute represents the action type for a particular element. Web page resources may be dynamic, with the same elements representing multiple resources based

on content. Web pages can either provide these as value annotations or set `data-ac4a-static` to false, requiring a configuration file for dynamic mapping.

```
{
    "https://outlook.live.com/calendar/0/view/workweek": {
        "verified": true,
        "read": [
          "div.ZlutZ"
        ],
        "write": [
          "span:contains('Save')"
        ],
        "create": [
          "span:contains('New event')"
        ],
        "data": {
          "Calendar:Year($data{button[aria-label*='select to change the month'] > span}[1])::
          Calendar:Month($data{button[aria-label*='select to change the month'] > span}[0])::
          Calendar:Day(?)": [
                "div.ZlutZ"
          ],
          "Calendar:Year($data{input[aria-label='Start date']}{split_slash}[2]@value)::
          Calendar:Month($data{input[aria-label='Start
          ↪  date']}{split_slash}[0](number_to_month)@value)::
          Calendar:Day($data{input[aria-label='Start date']}{split_slash}[1]@value)": [
            "span:contains('Save')"
          ],
          "Calendar:Year(?)": [
            "span:contains('New event')"
          ]
        }
    }
}
```

Fig. 4. Example web mapping configuration for Outlook calendar showing how web page elements are mapped to resources and actions. The configuration includes both static mappings (e.g., $*$) and dynamic mappings using the $data syntax to extract values from web page content.

3.5.2 Signature Instantiation. When an API function is called with concrete arguments, the dynamic permission signature is instantiated by evaluating the permission function with concrete values:

Definition 3.2 (Signature Instantiation). Given a permission function $M(\vec{a}) \longrightarrow (R, A)$ where R is the resource and A is the action, and a function call with concrete values $\vec{v}$, the instantiated dynamic permission signature is obtained by:

(1) Evaluating the permission function with the concrete argument values to get the resource $R(\vec{v})$ and action $A(\vec{v})$

The result is the instantiated signature $(R(\vec{v}), A(\vec{v}))$ where $R(\vec{v})$ and $A(\vec{v})$ represent the evaluation of the resource and action functions with arguments $\vec{v}$. The instantiated resource values and actions becomes the needed resources and actions in Algorithm 1.

3.5.3 Extraction and Enforcement. With appropriate permission functions in place, dynamic permission signatures can be extracted from various abstractions, though some approaches are more sensible or appropriate for the desired enforcement level. For example, dynamic permission signatures can be extracted directly from user input using an LLM and matched against existing permissions to verify whether a particular user request is authorized to access the desired resources. Similarly, dynamic permission signatures can be extracted from the intermediate output of the agent responsible for coming up with the execution plan describing the next step or complete roadmap of subtasks required to complete the user's task. This extraction can verify whether the plan is attempting to access resources not permitted by active permissions. Having dynamic permission signatures and the ability to extract them directly from natural language input from both the planner and user makes our framework extensible to agent implementations where implementing dynamic permission signature extraction is not trivial. However, this approach is less reliable because it uses LLMs, introducing a trust factor. We must trust the dynamic permission signature extractor to reliably protect against unintended resource access.

4 PERMISSION ENFORCEMENT

AC4A provides a permission management system that operates on the set of active permissions defined in the previous section. In principle, the system could provide a simple interface for adding and removing permissions directly as they are formally defined. However, to improve usability, *AC4A* offers a more convenient dashboard interface with multiple creation and management workflows. Management actions leverage the resource difference function (Section 3.4.1) to detect redundancy and, when an action is denied, to present the *Remaining* need as concrete candidate permissions the user may choose to add.

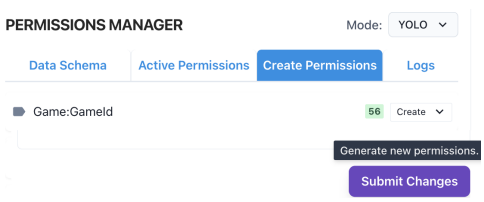

Fig. 5. Manual permission creation: Users specify data values and access types directly through the dashboard interface.

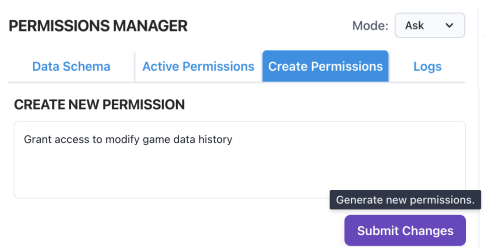

Fig. 6. LLM-assisted permission generation: Users describe access needs in natural language, which are then converted to formal permissions.

The system supports three primary modalities for permission creation, all of which ultimately add permissions to the active set. Manual creation allows users to interactively create permissions through the dashboard interface as shown in Fig. 5, which provides visual representation of the complete resource hierarchy, interactive selection of resource nodes and access types, real-time validation of permission syntax, and detection of redundancy using the resource difference function. This approach gives users full control over permission definition while providing a more intuitive interface than direct formal specification.

LLM-assisted generation enables natural language descriptions to be automatically converted to formal permissions shown in Fig. 6. Users can describe desired access in natural language, such as *grant access to calendar for July to see my schedule.* The LLM analyzes the request against the complete resource hierarchy and generates appropriate permissions. Users can review and modify generated permissions before they are added to the active set.

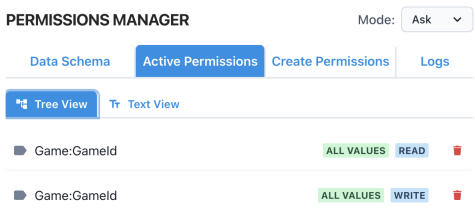

Fig. 7. Active permissions displayed in a structured, hierarchical interface for easy review and management.

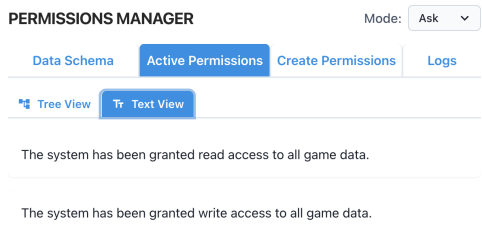

Fig. 8. Active permissions shown as natural language statements for improved readability and transparency.

For trusted users or planning systems, permissions can be inferred from user requests or execution plans. The system analyzes implicit access requirements in task descriptions and uses an LLM to generate natural language permission statements based on inferred needs. These natural language permissions are then given to the same pipeline which handles the manual creation of permissions from natural language.

The dashboard provides multiple views of active permissions. The structured view presents a hierarchical list showing data paths and access types as shown in Fig. 7, while the natural language view offers human-readable descriptions of permissions as shown in Fig. 8. The interface also includes a log view that shows the complete audit trail of all permission-related activities, including when permissions are created, removed, or used to grant or restrict data access. When an action is denied, the UI can display a compact summary of the *Remaining* need produced by the residual operator as guidance for what to add. This logging functionality provides transparency and accountability for all permission operations, enabling users to track how their access control permissions are being enforced over time.

The system attempts to reduce overlapping permissions but is not responsible for handling them if they exist. If a permission for an access is removed, the system does not guarantee that the access is not possible through another permission. The user generating permissions is responsible for handling this. For example, if read access to all 2025 calendar data exists, attempting to add a specific permission for October 2025 will be prevented as redundant because permission to a year will implicitly also grant access to all its months. However, if the October permission is created first, both permissions will coexist, with the system ensuring continued access even if the more specific permission is later removed.

The effectiveness of *AC4A* depends entirely on the quality of the defined permissions. While the framework provides robust enforcement mechanisms, the responsibility for determining appropriate permission sets lies with users and administrators. The system is designed to be orthogonal to permission decision-making, focusing solely on reliable enforcement of whatever permissions are defined. Applications may implement precise or conservative resource difference functions, and the UI reflects whatever *Remaining* the application computes.

4.1 Permission Handling Modes

Agents can be configured to handle permission failures gracefully or at least made aware of them to relay appropriate information to users. For instance, in our browser agent's system prompt, we explicitly mention that blocked areas on web pages with a 🚫 sign indicate insufficient permissions for accessing the resources. Similarly, in our API-based agent's system prompt, we add information about the type of error message that is returned when a call is not permitted due to insufficient

permissions. We support four modes of handling permission failures by directly modifying the agent's system prompt:

(1) **Ask:** Request the user to set the necessary permissions to continue and, if available, share the error with the user so they can configure appropriate permissions themselves.
(2) **Skip:** Suggest a workaround that does not require accessing the unauthorized resources or ask the user to suggest how to proceed.
(3) **Infer:** The system automatically infers permissions using an LLM in infer mode but does not deploy them without user approval. The agent then requests user approval for deploying the inferred permissions.
(4) **YOLO:** In YOLO mode, the system automatically infers and deploys permissions using an LLM. The agent is instructed to retry until the correct permissions are set automatically.

All generated and deployed permissions are logged in the interface along with their usage to allow or deny access to data. Thus, in any permission mode, users can monitor which permissions are being generated and used, particularly when an LLM is used to generate them.

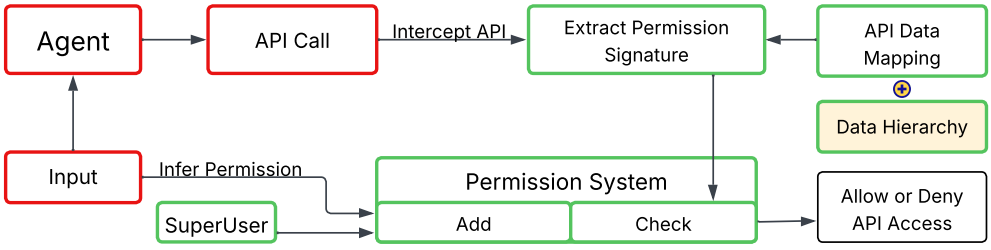

Fig. 9. Every API call that is made is intercepted and checked against the active permissions. If the call is not permitted, it is blocked and an error message is returned.

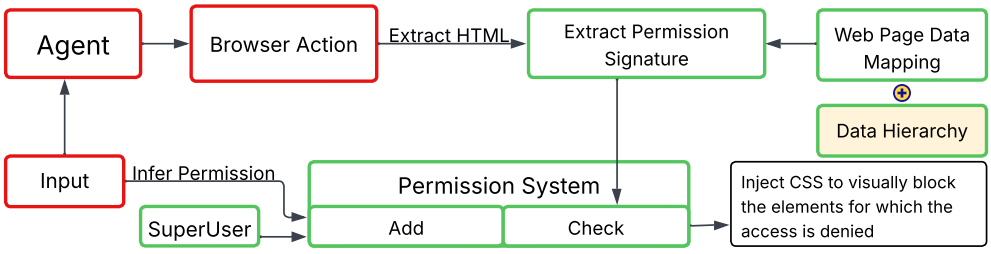

Fig. 10. After each browser action, the content of the web page is checked against the active permissions and the unauthorized content is replaced with a darkened area displaying a 🚫 sign before the screenshot is sent to the agent. This prevents agents from accessing unauthorized data.

4.2 Enforcing Permissions for API-based Agents

API-based agents operate by providing an LLM (capable of function or tool calling) with a list of tools that serve as wrappers over actual API calls, along with the schema for each endpoint and its

required arguments [4]. Based on the user query and system prompt, the LLM decides when to make a tool call, which eventually results in an API endpoint being invoked. To enforce permissions, we intercept API calls and block those that attempt to access resources not permitted by the active permissions, returning an error message when the call is not permitted, as shown in Fig. 9.

During runtime, each API call is intercepted and authorized via the resource_difference function. *AC4A* uses the permission checking algorithm described in Section 3.4 to iterate through all currently granted permissions and determine if they collectively cover the needed access.

If the iteration through all granted permissions results in an empty remaining set, the call is allowed. Otherwise, the call is blocked and returns a permission error along with a summary of what access is still needed.

For API-based agents, we deterministically extract permission signatures from API endpoints. API calls represent the lowest level of abstraction for agent resource access, making this approach more reliable. For instance, if we extract permission signatures from input or agent output but the actual API calls do not align with those signatures, we may fail to catch unauthorized access. However, enforcement at the lowest level ensures we can verify every possible resource access performed by the agent.

4.2.1 MCP Integration. Since many applications and their APIs have become standardized over time (e.g., Google Calendar, Expedia), it is inefficient for developers to write separate wrappers for each application. The Model Context Protocol (MCP) addresses this problem by creating a unified protocol for querying tools and tool schemas [11]. To add tools from a particular application to an agent, developers can directly use the application's MCP server to fetch the schemas of the tools they offer. This standardization allows schemas to be developed distributively or even crowdsourced while ensuring consistent usage across any agent. Since MCPs do not change how tool calls are made, they remain compatible with our permission enforcement mechanism. The resource schema and mapping used for extracting permission signatures can also be integrated with MCPs such that each application's MCP server hosts not only the schema for API endpoints but also the resource schema and mapping needed for permission signatures.

4.3 Enforcing Permissions for Browser-based Agents

Browser-based agents interact directly with web browsers using a simple abstraction over possible browser interactions such as clicking and scrolling. These interactions are provided as tools to the agent. Along with user input, such agents also receive a screenshot of the current web page. The underlying model analyzes the current state of the web page and decides what the next action should be, using tool calls to progress toward the goal. The same technique is also employed by OS-based agents [13], which can interact with multiple applications beyond just web browsers.

We enforce permissions over browser agents by ensuring that no part of the web page containing unauthorized data or interactive elements (such as buttons) that could lead to unauthorized actions is visible to the agent. For agents using screenshots to understand the state of the web page, we intercept the agent just before the screenshot is taken and inject CSS to replace elements whose mapped needs have nonempty Remaining with blocked areas displaying a 🚫 sign, as shown in Fig. 10. This ensures that the agent never sees the restricted elements on the web page.

While it's possible to implement access control for browser-based agents by extracting permission signatures from input or the generated plan, this approach is not sufficiently reliable. The lowest possible level of interaction consists of low-level operations like clicks and scrolls, and it is not trivial to deterministically map these operations to the data they enable interaction with. Furthermore, browser agents do not require any interaction mechanism to read data visible to them. Even if we implement access control over browser operations, agents will still have access to information

present on web pages when they load, before any browser operations are performed. For example, opening a calendar would require modeling web page opening as an action, significantly increasing complexity. Therefore, instead of performing access control at the level of low-level browser interaction actions, we implement it at a higher level, when screenshots are captured and sent to the agent for processing. This approach allows us to prevent unintended data access even before the agent gains access to the web page.

We intercept web pages after each action and before capturing screenshots that is later sent to the agent. The web page remains in the same state until the agent performs the next action. We extract the HTML of the web page in its current rendered state. Web pages may contain elements unrelated to resources (such as headings or logos) that we do not need to control access to, and these will always be displayed. We iterate over the mapping in our configuration file and generate permission signatures for each element using the provided mapping. For each element, we use the permission checking algorithm described in Section 3.4 to determine what access remains needed.

Only elements with nonempty Remaining are blocked by injecting CSS to replace their content with a darkened area displaying a 🚫 sign.

Like API-based agents, we require the resource schema to be provided for web pages, which is the same as that provided for APIs. However, access control for browsers is more challenging than for API-based agents because web pages consist of non-data elements we do not control, data that can be read/viewed by agents, and interactive elements that lead to access to additional data (such as search or send buttons, which require read and write access respectively).

5 PROTOTYPE IMPLEMENTATION

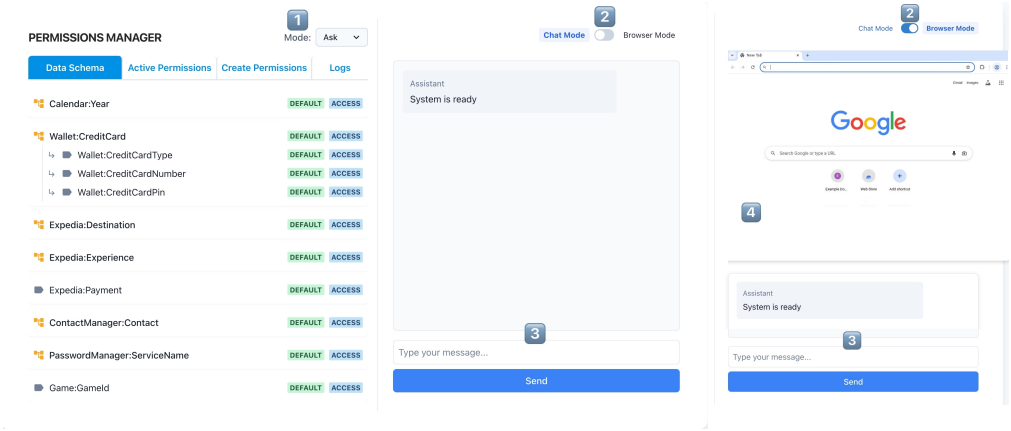

Fig. 11. *AC4A* interface showing both API-based agent (middle) and Browser-based agent (right). On the left is the permission dashboard, with the *Data Schema* tab displaying all available resources from every application registered with *AC4A*. The *Active Permissions* tab lists currently active permissions, *Create Permissions* allows users to add new permissions, and *Logs* records the creation, deletion, and usage of permissions, including which permissions allow or block access. (1) lets the user select the permission mode used by the agent to handle permission errors; (2) allows switching between chat mode and browser mode; (3) is the chat interface; and (4) shows a live preview of the browser in browser-agent mode.

We have implemented a prototype with a permission dashboard for creating permissions using both text and the UI as shown in Fig. 11. The active permissions are listed as UI objects and as text, along with an option to delete them. The Logs tab records all permissions created, deleted, and

used for granting access. It also provides an option to select the permission mode for automatically inferring permissions from user input and the agent's intermediate plan responses (YOLO and Infer), or for manually handling permission errors (Ask and Skip). Although LLM-based generation is highly efficient, it cannot always be trusted; users can choose to use it and monitor the generated permissions. In the permission dashboard, all resources from all applications registered with *AC4A* are available for exploring or creating permissions. The main component is a chatbot backed by an Autogen-based agent system [30], which has access to common application endpoints like a calendar. These endpoints are either backed by LLM-based mock responses that simulate API endpoint responses or are real API endpoints. Each application provides a mapping from API endpoints and arguments to the resource hierarchy, which *AC4A* can access to generate permission signatures. The same permissions are enforced simultaneously in both API- and browser-based agents.

Our interface also has an option to switch to browser mode, which shows the browser state in the chat interface and can be accessed directly by hovering over the preview, turning it into a complete browser. We run an instance of Chromium browser and expose it through a noVNC server. The previews are the latest screenshots of the webpage state. For both API- and browser-based agents, we use GPT-4.1. In the prototype, we do not employ a computer-use model, but instead use GPT-4.1 to generate actions as text, which we then manually execute. This approach does not affect *AC4A*'s functionality, and the computer-use model can be replaced with some plumbing to connect it to the browser using Playwright.

5.1 Design Demonstration: Tic-tac-toe Game Example

In this section, we demonstrate the end-to-end operation of *AC4A* using a simple tic-tac-toe game website with exposed backend APIs. This example illustrates how the framework's core components work together to protect unauthorized resource across in different agent implementations.

5.1.1 Resource modeling and Mappings. To enable any webpage or application to work with *AC4A*, we must model resources that needs to be protected against unauthorized access. For our tic-tac-toe game, we want to implement access control over game history data to prevent agents from viewing or deleting it without proper authorization. We model the resource type as $Game : GameId$ without sub-resources for simplicity. While we could create more granular sub-resources such as $Game : GameId : GameStatus$ or $Game : GameId : GameResult$ for finer access control (e.g., preventing access to games the player lost), we keep the model simple with just $GameId$.

The application must also provide mappings between API endpoints and resources, along with the type of data access. These mappings must capture the endpoint and arguments regardless of their values, as they will be evaluated at runtime for calculating permission signatures. Consider two endpoints: `get_games()` and `delete_game(game_id)`. We map `get_games()` to resource $Game : GameId(?)$ because it returns all games and requires access to all games, with a read access type. For `delete_game(game_id)`, we map it dynamically to resource $Game : GameId(game_id)$, where $game_id$ is resolved at runtime. For instance, if `delete_game` is called with id 45, the mapped resource becomes $Game : GameId(45)$, with a write access type since deletion modifies the game data.

For browser-based agents, we create mappings between HTML elements and resources. Since we only want to control access to game history data, we map the complete game history element to $Game : GameId(?)$ with read access type, allowing viewing of the complete game history. For delete operations, we require write access. The API mapping function is shown in Figure 12, while the web element mapping configuration is shown in Figure 13. The read, write, and create lists specify elements that allow read, write, and create access respectively, while each resource item

lists the related elements from the resource types. With these mappings established for both API and browser interfaces, our permission system can control access for API-based, browser-based, or hybrid agents.

```python
def generate_attributes(self, kwargs, endpoint_name, wildcard):
    game_id = kwargs.get('game_id', '?')
    if wildcard:
        game_id = '?'

    if endpoint_name == 'get_games':
        granular_data = f"Game:GameId(?)"
    elif endpoint_name in ['get_game', 'delete_game']:
        granular_data = f"Game:GameId({game_id})"
    else:
        granular_data = f"Game:GameId({game_id})"

    data_access = 'Write' if 'delete' in endpoint_name else 'Read'

    return {
        'granular_data': granular_data,
        'data_access': data_access
    }
```

Fig. 12. API mapping function for the tic-tac-toe game demonstrating how different endpoints are mapped to the resources based on their arguments and access types. The function generates permission signatures by mapping endpoint names and arguments to specific resources with wildcard support for operations that access all games.

```json
{
    "read": [
        "#historyList"
    ],
    "write": [
        "button:contains('Delete')"
    ],
    "Game:GameId(?)": [
        "#historyList",
        "button:contains('Delete')"
    ]
}
```

Fig. 13. Web element mapping configuration for the tic-tac-toe game showing how HTML elements are mapped to the resources. The configuration defines read/write/create access lists and resource-specific element mappings that enable fine-grained access control for browser-based agents.

5.1.2 Access Control Enforcement: No Permissions Active. Initially, with no active permissions, attempting to access game history by instructing the agent to retrieve game history results in a tool call that would invoke the corresponding API (in this case, `get_games`). *AC4A* intercepts these

calls and uses the mapping to extract a permission signature, in this case, $Game : GameId(?)$ with read access. The system computes Remaining by subtracting the active read-permissions from this need using the resource difference function. Since no permissions exist initially, Remaining is not empty and access is denied, as shown in Figure 14.

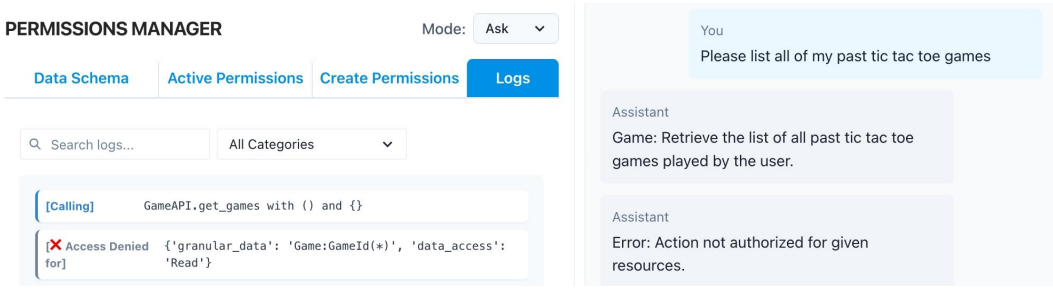

Fig. 14. (Left) API access log showing permission signature extraction and access denial for the `get_games` endpoint when no permissions are active. (Right) Excerpt from the chat showing permission error.

For browser-based agents, when requesting the agent to look up tic-tac-toe game history, it first instructs to open the game webapp. *AC4A* intercepts the browser agent's actions and fetches the configuration file containing HTML element mappings. The system creates permission signatures for each resource type in the configuration and computes Remaining against the active permissions. Elements whose Remaining is not empty are blocked with a solid dark box that displays a 🚫 sign, informing the agent of restricted access. This process completes before the next screenshot is captured and provided to the agent. The screenshot is shown in Figure 15.

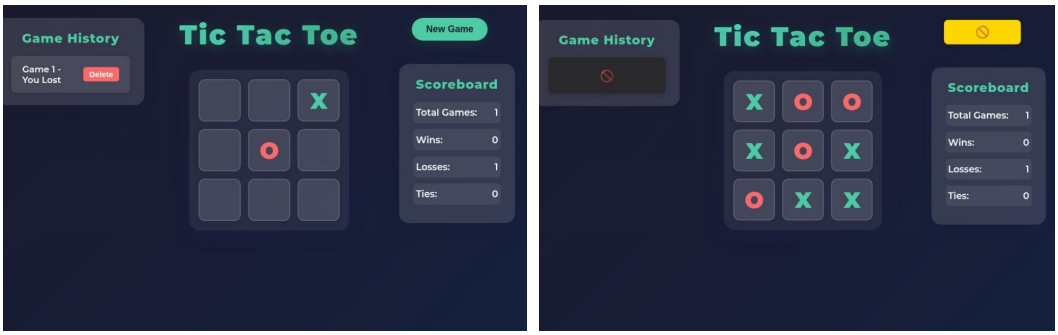

Fig. 15. (Left) Browser screenshot before applying permissions showing all the elements that are not shown to the agent. (Right) Browser screenshot showing the tic-tac-toe game interface with game history elements blocked by solid color boxes (black for read; red for write; yellow for create) and prohibited signs when no permissions are active and is shown to the agent.

5.1.3 Partial Access: Read Permissions Only. Suppose we now add a read permission for all game data, enabling agents to access game data while preventing deletion of game history. The permission signature generated for API-based agents attempting to delete games will require write access to game history data, but since no permission allows write operations, the request fails due to a permission conflict as shown in Figure 16.

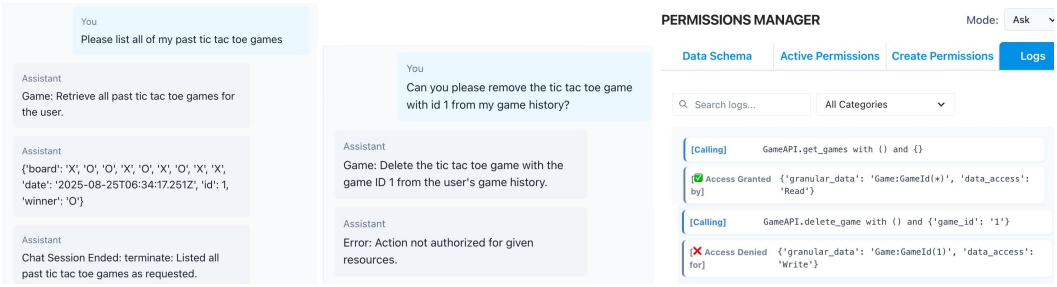

Fig. 16. (Left) Excerpt from chat showing agent successfully reading the game data after granting the read permission. (Center) Excerpt from chat showing agent failing to delete game history data because of missing write permissions. (Right) API access log showing permission signature extraction and access denial for the `delete_game` endpoint due to missing write permission but successful call to `get_games` endpoint due to read permission.

For browser-based agents, with read permissions now active, the game history becomes visible. However, areas containing delete game buttons remain visually blocked because no permission exists for deleting or modifying game history through write data access. The screenshot is shown in Figure 17.

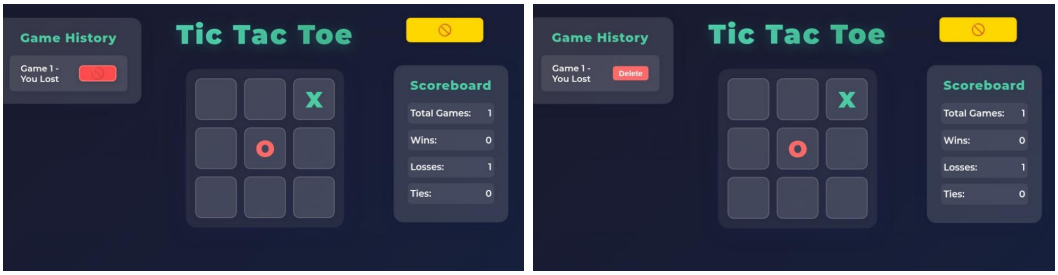

Fig. 17. (Left) Browser screenshot after applying the read permission and before granting the write permission. The game history is now visible but the delete button is not visible to the agent. (Right) Browser screenshot after both read and write permissions. The agent can now access the delete button.

5.1.4 Full Access: Read and Write Permissions. When both read and write permissions are active, agents can successfully access and modify game data. The permission signature extraction process now finds matching permissions for both read and write operations and allows the agent to delete games as shown in Figure 18.

For browser-based agents, with both read and write permissions now active, the game history becomes visible and the delete buttons are accessible. The screenshot is shown on the right side of Figure 17.

This example demonstrates our approach with a single API backend and a simple web app. Real-world web pages and APIs are significantly more complex, comprising multiple elements connected to various data types. We have successfully scaled this technique to realistic use cases such as Outlook Calendar, Expedia, and Trello using more complex mappings to extract permission signatures

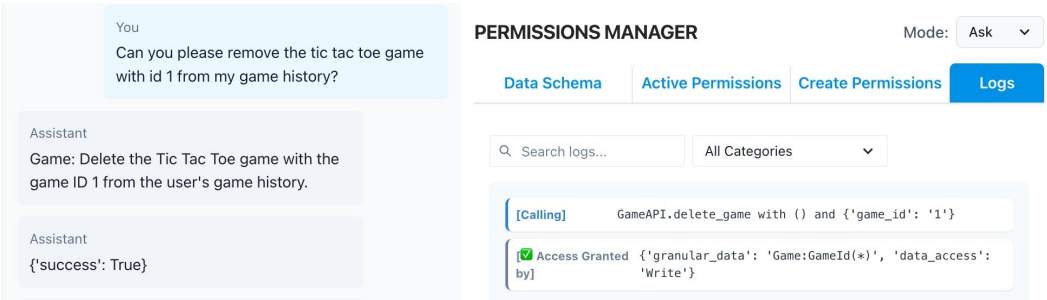

Fig. 18. (Left) Excerpt from chat showing agent successfully deleting the game data after granting the write permission. (Right) API access log showing permission signature extraction and access granted for the `delete_game` endpoint.

6 DISCUSSION

6.1 Access Control at Different Levels in Agent Pipeline

The abstraction level at which access control operates is crucial for several reasons. If implemented at a higher level, we lose control over lower-level operations that might pose safety risks. For example, *AC4A* can be configured to work at any level of abstraction in a given agent pipeline, but implementing it at the lowest possible level provides stronger guarantees about preventing unauthorized access. However, there may be future domains where extracting permission signatures is not possible or feasible, such as browser operations like clicking and scrolling. In such cases, the input, the execution plan, or other intermediate stages can be used to extract permission signatures. The choice between ease and feasibility of abstraction for access control versus the guarantees it provides is not directly related and may well depend on the method used for extracting permission signatures.

That said, the permission signature and permission infrastructure we have developed is generic enough that any part of the agent pipeline can be brought under its control using a correct and reliable method for extracting permission signatures.

6.2 Adaptation to Variants of API- and Browser-Based Agents

In this paper, we demonstrate how *AC4A* can be used for access control of resources for API-based and browser-based agents. However, it can also be specialized and implemented for specific API-based agents, such as code assistant agents like Cursor, where the resources become access to files and directories. *AC4A* can be used to protect against unintended accesses, such as deleting important files.

Similarly, browser-based agents that use not just screenshots but also CSS selectors for improved reliability and context can be easily supported. When operations involve clicking on specific CSS selectors, since our mapping already exists between CSS selectors and data, we can use the exact same mapping and intercept these calls directly instead of intercepting the browser state before taking screenshots. This allows us to directly block calls that interact with the browser at the CSS selector level.

6.3 Unavailable Mapping / Not Mapped

Correct mapping is important for effectively performing access control over the resources accessed by the agent. If an API is not mapped when the permission signature is extracted, a default signature is returned. It is then up to the application writer to define this default. In most cases, it makes

sense to link unmapped APIs to all resources, meaning their permission signature grants access to all resources.

APIs are typically written to interact with resources, and those that do not can be mapped to permission for any resource. *AC4A* requires an active permission to grant access for any permission signature.

Similarly, for webpages, each element might not be related to resources, for example, headings or titles. One could argue that these elements are also not required for the agent's operation, but it would be very challenging to create a mapping that covers all elements on a webpage. Therefore, the configuration file only includes elements that contain resources, and all other elements fall outside the scope of *AC4A*.

If we do not use a configuration file but instead rely on custom annotations supported by the webpage for *AC4A*'s operation, then we need not worry about missing elements because we would expect all resources elements to be correctly annotated.

6.4 Modeling Resources for Finer Access Control

Unintended accesses that we care about are always related to a resource access, but there are cases where we have very rich permissions or very specific case we want to restrict. In such cases, the resource hierarchy can always be extended to support richer hierarchies. For example, allowing access to cookies versus allowing access to essential cookies. To support the first case, we need to model cookies as resources, but for the second case, we need to differentiate between cookies and essential cookies such that the latter is a parent of the former. Most resources exist in hierarchies, and our hierarchies can be extended without breaking existing permissions.

6.5 API + Browser Agents

API and browser agents can coexist entirely. We are seeing browser being available as an MCP [2] that can be called by API-based agents when access to a particular API is not available. In such cases, existing access control frameworks that only work on API arguments will not be able to perform access control on actions taken by the agent in the browser. However, *AC4A*, having a unified data representation and a common permission infrastructure, can effectively perform access control on data accesses through both APIs and browsers.

6.6 Finding Good Permissions

AC4A provides flexible ways for creating permissions both manually and automatically. It can therefore be used to evaluate methods for creating permissions. For example, user-driven access control [22] relies on inferring implicit permissions from user intent. *AC4A* can act as a testbed for evaluating the effectiveness of multiple such permission control methods, and the existing permission generator in *AC4A* can also be used as a baseline.

7 LIMITATIONS

AC4A is an access control framework for enforcing permissions over resources which works independently of the agent implementation. While there are significant strengths in having such a resource-centric design for access control, there are also limitations that we discuss in this section.

7.1 Mapping File Requirements and Maintenance

We consider the requirement of either mapping files to extract permission signatures or annotations as one of the most significant limitations. Writing these mapping files can be challenging, and they require constant maintenance to match the current state of webpages or API endpoints. It is also

possible to make mistakes during this process, which might lead to false positives or negatives in permission enforcement.

7.2 Complexity in Browser Mapping Definitions

The way we define mappings, particularly for browsers, is complex but can still be improved by allowing more programming constructs such as conditional statements (if-then) dependent on the data present on the page, or unbounded loops to cover multiple possible child elements that become bounded at runtime based on the current state of the HTML. We consider making the DSL more expressive in the future.

7.3 Context-Dependent Automatic Permission Extraction

Sometimes permissions cannot be directly extracted. For example, a *create new event* button allows users to create new events, but its permission signature does not necessarily equate to write access to all events. These determinations are context-dependent, and while webpage developers can model them effectively, they may prove difficult for automated permission extractors to handle accurately.

7.4 Limitations in Modeling Complex Access Control Requirements

There are access control requirements that may not be easily modeled within this framework. For instance, implementing access control over different ranges of data values, such as allowing checkout access only when the amount is less than 200 USD, presents challenges. To achieve this, one might need to model *checkout* and *checkout under 200 USD* as separate subdata entities. In future work, we plan to include conditionals in permissions that will not only allow values for data but also operations over the value of the data. This will enable the creation of permissions such as *access to checkout with data less than 200.* We plan to support conditions in our permissions in the future.

7.5 Scope Limitations

AC4A focuses exclusively on implementing access control using permissions. While it does provide working methods for automatically generating permissions or inferring them from user input, this functionality is not the primary focus and falls outside our current scope. This limitation motivates future research directions regarding what constitutes good permissions for agents and what permissions users actually intend to grant.

7.6 Persistence and Permission Lifecycle Management

Another significant limitation concerns the persistent nature of permissions. Consider a scenario where a user grants write permission to create an event, and the agent maliciously uses the same permission later to delete another event. One potential solution involves implementing single-use permissions that are removed once they allow an access. This falls more under the domain of accessibility considerations, which we do not currently handle, and raises questions about who should decide whether permissions must remain active. Additionally, we cannot guarantee that removing a permission will restrict all access, as there might be other permissions that could allow the same access, or alternative ways to bypass restrictions, particularly in web browser setups.

8 RELATED WORK

Agents bring new challenges to access control systems and though none of the existing techniques are directly applicable but have greatly influenced the design of techniques proposed to address these challenges. In this section, we discuss the existing access control systems and the techniques

proposed to address the challenges brought by agents. To our best knowledge, *AC4A* is the first framework to perform access control over resources and for browser-based agent interactions let alone being the first that uses the same permission representation for both API- and browser- based agents similar to file hierarchy from UNIX file system [21] and the unified data representation in Zanzibar [20].

8.1 Traditional Access Control Systems

Traditional access control systems have evolved from simple file-based permissions to sophisticated capability-based models [10]. Unix systems established the foundational concept of permission signatures through file-based access control using permission bits, specifying read/write/execute privileges for user/group/others using octal notation. These systems introduced identity-based enforcement where permissions are tied to user/group identities, static permission models with fixed privileges assigned at resource creation, and binary operations for permission modification. While these systems established explicit permission declarations, they lacked flexibility for dynamic environments and complex policy requirements. *AC4A*'s data hierarchy is similar to the data hierarchy in UNIX file system along with read and write permissions.

Mobile platforms and web browsers extended this model through declarative permission signatures that specify required capabilities upfront [18]. Android's permission system represents a significant evolution with manifest-based declarations, protection levels categorized as normal, dangerous, signature, or signatureOrSystem, and runtime permission models requiring explicit user consent for dangerous permissions [6]. Modern web browsers implement permission signatures through standardized APIs, including the Permissions API for querying permission states without triggering user prompts [5], origin-based scoping where permissions are granted per-origin with explicit user consent, and capability-based access for geo-location, camera, and microphone through standardized interfaces [17]. *AC4A*'s extends the concept of permission signature to the agents that is dynamically inferred from the tentative interactions with the agent.

Access control models have evolved to address increasingly complex security requirements. Role-Based Access Control (RBAC) systems implement permission signatures through role definitions with explicit associations between roles and capabilities [9], hierarchical structures enabling inheritance-based permission propagation, and separation of duties requiring multi-role participation for sensitive operations. Attribute-Based Access Control (ABAC) extends this through dynamic attribute evaluation with multi-attribute policies considering subject, object, and environment attributes [12], boolean policy functions for complex rule evaluation, and context-aware decisions enabling dynamic permission granting based on runtime conditions. *AC4A*'s permission model is similar in a lot of ways like the propagation of permissions through inheritance and the use of boolean policy functions for complex rule evaluation. Zanzibar is a pioneer work in developing a unified way for enforcing access control policies across different services. *AC4A*'s permission model has parallels in Zanzibar in that it uses a unified way for enforcing permissions over same data across different agent implementations.

8.2 Access Control for AI Agents

Recent work has focused on protecting agents from adversarial threats through both secure system design and enforceable policies. While *AC4A* does not directly address adversarial threats, it provides a framework for managing permissions across different agent implementations. That said, *AC4A* can be used to build secure systems for agents by providing appropriate permissions and restricting access to unintended data.

In related work, IsolateGPT proposes isolating agents via a hub-spoke architecture, connecting them to a central planner [31]. CAMEL monitors deviations from trusted intra-agent control

and data flows extracted from user queries [8]. Similarly, ACE [16], F-Secure [29], and FIDES [7] extract policies, execution plans, or feasible data/control flows statically for runtime enforcement. FIDES specifically constrains inter-agent flows, while ACE and F-Secure target intra-agent integrity. Collectively, these approaches focus on information flow control to prevent indirect prompt injection by ensuring flows adhere to intended paths. While this overlaps with preventing unintended data access, it specifically addresses flow hijacking rather than raw data protection.

There are specialized access control systems for agents specifically, such as Progent [24] and Conseca [26], but both focus on creating policies over actions with constraints on the arguments those actions/tool calls can have. *AC4A* is the first framework that provides a unified, data-focused framework for heterogeneous agent environments. This means it creates permissions over data rather than actions, which works not only for API-based tool calls but also for browser-based agent interactions.

9 CONCLUSION

As LLM agents become more autonomous, their coarse-grained access control models present significant challenges. In this paper, we introduced *AC4A*, a novel framework that provides fine-grained access control for both API and browser-based agents. Our core contribution is a resource-centric approach that enforces permissions on hierarchical resource structures. Through our evaluation, we show that *AC4A* effectively prevents unauthorized resource access in realistic scenarios involving real world applications.

REFERENCES

[1] Cursor - The AI Code Editor — cursor.com. https://cursor.com/en. [Accessed 03-07-2025].

[2] GitHub - microsoft/playwright-mcp: Playwright MCP server — github.com. https://github.com/microsoft/playwright-mcp. [Accessed 04-07-2025].

[3] Introducing Operator — openai.com. https://openai.com/index/introducing-operator/. [Accessed 03-07-2025].

[4] OpenAI Platform — platform.openai.com. https://platform.openai.com/docs/guides/function-calling?api-mode=responses. [Accessed 03-07-2025].

[5] Permissions API - Web APIs | MDN — developer.mozilla.org. https://developer.mozilla.org/en-US/docs/Web/API/Permissions_API. [Accessed 04-07-2025].

[6] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. Pscout: analyzing the android permission specification. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 217–228, 2012.

[7] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing ai agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[9] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. Role-based access control (rbac): Features and motivations. In *Proceedings of 11th annual computer security application conference*, pages 241–48, 1995.

[10] Pavlo Hlushchenko and Valerii Dudykevych. Exploratory survey of access control paradigms and policy management engines. *CMIS 2024 Computer Modeling and Intelligent Systems 2024*, pages 263–279, 2024.

[11] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions, 2025.

[12] Vincent C Hu, D Richard Kuhn, David F Ferraiolo, and Jeffrey Voas. Attribute-based access control. *Computer*, 48(2):85–88, 2015.

[13] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, et al. Os agents: A survey on mllm-based agents for computer, phone and browser use, 2024.

[14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, January 2025.

[15] Sangyeop Kim, Yohan Lee, Yongwoo Song, and Kimin Lee. What really matters in many-shot attacks? an empirical study of long-context vulnerabilities in llms, 2025.

[16] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. Ace: A security architecture for llm-integrated app systems, 2025.
[17] Tongbo Luo and Wenliang Du. Contego: Capability-based access control for web browsers: (short paper). In *International Conference on Trust and Trustworthy Computing*, pages 231–238. Springer, 2011.
[18] Sardasht Mahmood, Bakhtiar Amen, and Rebwar M Nabi. Mobile application security platforms survey. *International Journal of Computer Applications*, 133(2):40–46, 2016.
[19] Mykyta Mudryi, Markiyan Chaklosh, and Grzegorz Wójcik. The hidden dangers of browsing ai agents, 2025.
[20] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, et al. Zanzibar:{Google's} consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 33–46, 2019.
[21] Kristal T Pollack and Scott A Brandt. Efficient access control for distributed hierarchical file systems. In *22nd IEEE/13th NASA Goddard Conference on Mass Storage Systems and Technologies (MSST'05)*, pages 253–260. IEEE, 2005.
[22] Franziska Roesner, Tadayoshi Kohno, Alexander Moshchuk, Bryan Parno, Helen J. Wang, and Crispin Cowan. User-driven access control: Rethinking permission granting in modern operating systems. In *2012 IEEE Symposium on Security and Privacy*, pages 224–238, 2012.
[23] Charles Salemi, Suryakanta Shah, and Eric Lund. A privilege mechanism for {UNIX} system v release 4 operating systems. In *USENIX Summer 1992 Technical Conference (USENIX Summer 1992 Technical Conference)*, 1992.
[24] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for llm agents. *arXiv preprint arXiv:2504.11703*, 2025.
[25] Susan Stepney and Stephen P Lord. Formal specification of an access control system. *Software: Practice and Experience*, 17(9):575–593, 1987.
[26] Lillian Tsai and Eugene Bagdasarian. Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HOTOS '25, page 8–17. ACM, May 2025.
[27] Bibek Upadhayay, Vahid Behzadan, and Amin Karbasi. Cognitive overload attack:prompt injection for long context, 2024.
[28] Haowei Wang, Junjie Wang, Xiaojun Jia, Rupeng Zhang, Mingyang Li, Zhe Liu, Yang Liu, and Qing Wang. Adinject: Real-world black-box attacks on web agents via advertising delivery, 2025.
[29] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024.
[30] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
[31] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. Isolategpt: An execution isolation architecture for llm-based agentic systems. *arXiv preprint arXiv:2403.04960*, 2024.
[32] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023.
[33] Baolei Zhang, Haoran Xin, Minghong Fang, Zhuqing Liu, Biao Yi, Tong Li, and Zheli Liu. Traceback of poisoning attacks to retrieval-augmented generation, 2025.
[34] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. 2024.